Forsterkende læring (Reinforcement learning)

IN1160 - Seminar

Vår 2026

Hjernetømming

Hva husker du fra forelesningen?

- Videoen i forelesningen viser agentene som lærer seg “Hide and Seek”. Den forklarte at forsterkende læring (reinforcement learning) er inspirert av hvordan dyr lærer.

Skriv i 3 minutter uten å stoppe

- Hva menes med at forsterkende læring er inspirert av dyr?
- Hva husker du ellers fra videoen og forelesningen?
- Ikke tenk for mye, bare skriv (Ingen behøver å lese det du skriver)

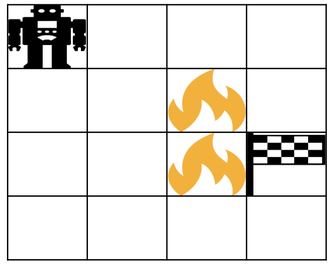
Nøkkelbegreper

Jobb i par (7 min)

- Tenk selv i 1 minutt, så jobb sammen med å skrive deres egne definisjoner av:
  - Agent (agent) Den som tar beslutninger, observerer miljøet, velger en handling, mottar belønning .
  - Miljø (environment) Verdenen agenten befinner seg i. Definerer hvordan handlingene påvirker tilstanden og hvilke belønninger agenten får.
  - Tilstand (state) Informasjonen agenten har om omgivelsene akkurat nå.
  - Handling (action) Hva agenten kan gjøre i miljøet.
  - Belønning (reward) Et tall som sier om handlingen var god eller dårlig.

Forsterkende læring i tre spill

- I forsterkende læring (reinforcement learning) lærer en **agent** ved å samhandle med et miljø. Agenten observerer en **tilstand**, velger en **handling**, og får en **belønning** fra miljøet. Dette skjer gjentatte ganger mens agenten prøver å maksimere total belønning.



a) Todimensjonal labyrint

b) Dataspillet "*Snake*"

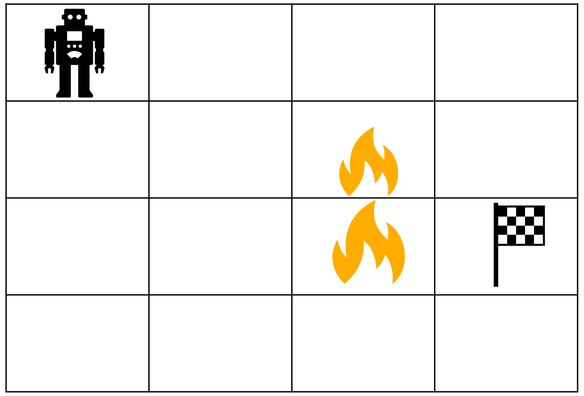
Figure 7. Chess game Graphical User Interface (GUI).

c) Sjakk

Hvem eller hva er agenten?
Hva er miljøet?

Agenten og miljøet

- **Todimensjonal labyrint:**
  - Agent: Robotspilleren
  - Miljø: Hele labyrinten: rutenett, flammer, vegger, flagg



- **Snake:**
  - Agent: Slangen (hodet)
  - Miljø: Hele spillebrettet (slange, vegger, epler, egen kropp)

- **Sjakk:**
  - Agent: Sjakkspilleren som velger trekk
  - Miljø: Sjakkbrettet (8x8 felter, brikker, lovlige og ulovlige trekk)

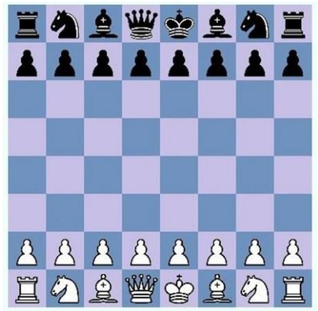


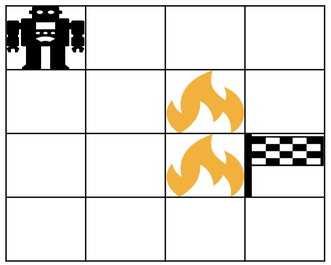
Figure 7. Chess game Graphical User Interface (GUI).

Å lage et veldefinert miljø

- For at en agent skal lære må miljøet være presist definert med **tilstander** (states), **handlinger** (actions) og **belønninger** (rewards), og representert med tall, lister, eller strenger.

Jobb i par (10 min)

For hvert spill:
1. Definer hva tilstander, handlinger og belønninger kan være.
2. Hvordan kan tilstandene, handlingene og belønninger representeres i en datamaskin?



a) Todimensjonal labyrint

b) Dataspillet "*Snake*"

Figure 7. Chess game Graphical User Interface (GUI).

c) Sjakk

Å lage et veldefinert miljø

a) Todimensjonal labyrint

- **Tilstand:** (x,y), eller én indeks, eller et full 2D-array
- **Handling:** 0=opp, 1=høyre, 2=ned, 3=venstre
- **Belønning:** +1 flagg, -1 flamme, -0.01 per steg

b) Snake

- **Tilstand:** piksler eller posisjoner (slangehode + eple)
- **Handling:** 0=opp, 1=høyre, 2=ned, 3=venstre
- **Belønning:** +100 eple, -1 steg, -20 kollisjon

c) Sjakk

- **Tilstand:** 8x8-array med brikkekoder (“hb” = hvit bonde)
- **Handling:** Fire tall (fra-til koordinater på sjakkbrettet). Vi kan også fjerne ulovlige trekk fra handlingene, eller gi dem stor negativ belønning.
- **Belønning:** +100 sjakk matt, -100 tap, +3 for å vinne en brikke, -3 for å tape en brikke, 0 ellers

Chatbot for pizzabestilling

- I dette tilfellet er ikke miljøet et fysisk spillebrett, men en interaksjon med en bruker som ønsker å bestille pizza. Chatboten må stille riktige spørsmål, tolke svarene, og til slutt gjøre en bestilling.

- Chatboten kan svare med noen få forhåndsdefinerte setninger:

1. "Hvilken størrelse vil du ha?"
2. "Hvilke toppings ønsker du?"
3. "Hva er adressen din?"
4. "Beklager, jeg forstår ikke. Kan du si det på en annen måte?"
5. "Da er bestillingen registrert!"

Jobb i par (8 min)

- Definer:
  1) Agent
  2) Miljø
  3) Tilstander
  4) Handlinger
  5) Belønning

- Foreslå hvordan tilstandene, handlingene og belønningene kan representeres i en datamaskin.

- Diskuter hvorfor dette er forsterkende læring.

Chatbot for pizzabestilling

- **Agent:** Chatboten (dialog manager). Velger neste handling og har tilgang til dialogtilstanden
- **Miljø:** Brukeren, brukerens svar, dialoghistorikken, eksterne regler (hva slags informasjon som må være på plass før bestillingen kan registreres)
- **Tilstander:** Den interne kunnskapen om hva som er oppnådd så langt.
- **Handlinger:** Agenten velger én av et definert sett dialoghandlinger
- **Belønning:**

```
state= [
    has_size, // 0 eller 1
    has_address,  // 0 eller 1
    has_topping, // 0 eller 1
    dialog_step // heltall
]
```

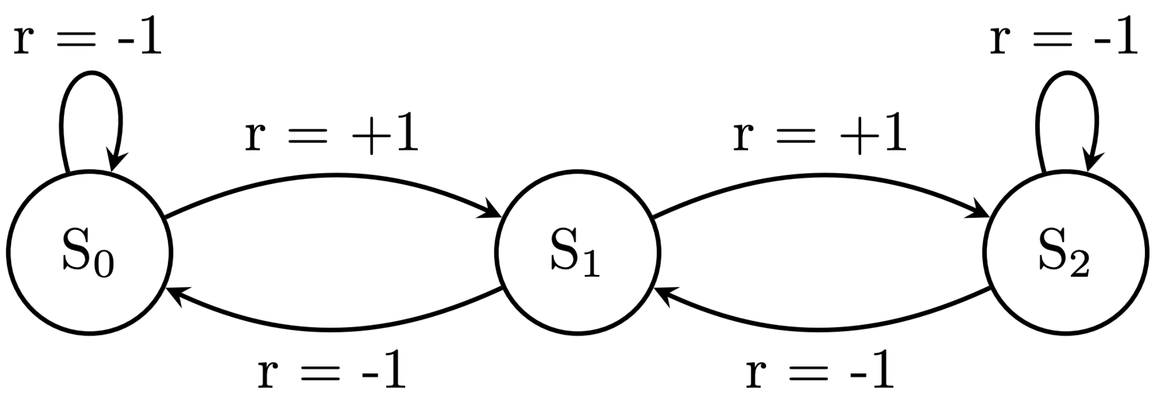
Representeres som en liste med tall: [1, 1, 0, 4]

+10 bestilling klar(all informasjon er hentet inn)
+1 hver gang informasjon fylles inn
-1 for “jeg forsto ikke” meldinger
-10 hvis agenten fullfører før informasjonen er komplett
-5 hvis brukeren avbryter interaksjonen

1. Spør om størrelse
2. Spør om topping
3. Spør om adresse
4. Beklag misforståelse
5. Fullfør bestilling

Læringsalgoritmer

SARSA og Q-læring



I hver tilstand er det to handlinger, agenten kan bevege seg til høyre eller venstre. Pilene i figuren viser hvilken tilstand man ender opp i og hvilken belønning som blir mottatt. Dersom man går til venstre i tilstand S_0 eller til høyre i tilstand S_2 ender man opp i samme tilstand.

a) Anta at agenten er i tilstand S_1 og går til høyre. Hvilken tilstand ender agenten opp i og hvilken belønning mottar den?

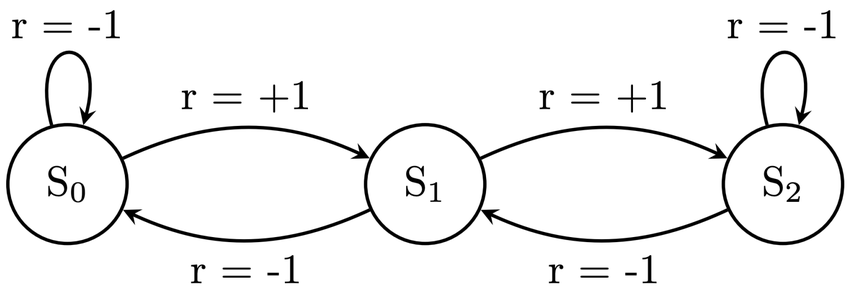
S_2 og belønning +1

b) Anta at agenten er i tilstand S_2 og går til venstre. Hvilken tilstand ender agenten opp i og hvilken belønning mottar den?

S_1 og belønning -1

Læringsalgoritmer

SARSA og Q-læring



$$Q = \begin{bmatrix} 1 & 0.5 \\ -1 & 1 \\ -2 & -1.5 \end{bmatrix}$$

Vi har en agent med følgende midlertidig **Q-tabell**, der hver rad tilsvarer en tilstand, henholdsvis tilstand S_0, S_1 og S_2, og hver kolonne tilsvarer et valg, henholdsvis venstre og høyre. Her er for eksempel Q-verdien til å være i tilstand S_0 (første rad) og gå til venstre (første kolonne) Q[0, 0] = 1 (med indeksering som starter på 0), mens Q-verdien for å være i tilstand S_2 (tredje rad) og gå til høyre (andre kolonne) er Q[2, 1] = −1.5.

Q-tabell-oppdateringsregelen for **SARSA**: $Q[S,A] \leftarrow Q[S,A] + \alpha(R + \gamma Q[S',A'] - Q[S,A])$

Q-tabell-oppdateringsregelen for **Q-læring**: $Q[S,A] \leftarrow Q[S,A] + \alpha(R + \gamma \max_a Q[S',a] - Q[S,A])$

Her er nåværende tilstand og handling henholdsvis S og A, som gir belønningen R og den nye tilstanden S'. Fra denne tilstanden vil agenten gjøre handlingen A'. For Q-læring, er $\max_a Q[S', a]$ handlingen a fra tilstanden S' som har høyest Q-verdi. Sett læringsraten **α = 0.5** og diskontinueringsfaktoren **γ = 0.9.** Anta nå at agenten er i tilstand $\mathbf{S_0}$ og går først til **høyre** og deretter til **venstre**.

Beregn først individuelt og sammenlign deretter parvis:

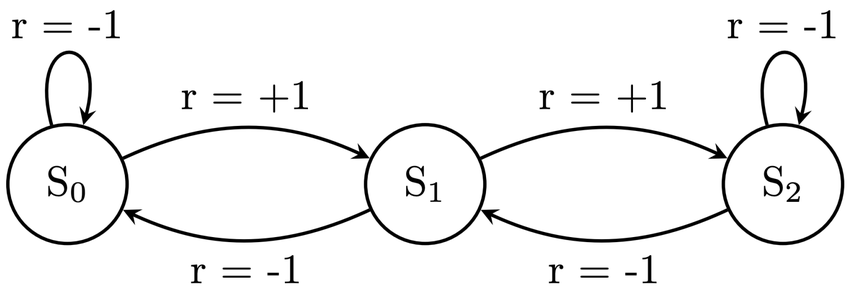
c) Bruk oppdateringsregelen til SARSA til å oppdatere Q-tabellen for tilstand S_0 og handling "høyre". Q[S,A] = Q[0,1] = 0.3

d) Bruk oppdateringsregelen til Q-læring til å oppdatere Q-tabellen for tilstand S_0 og handling "høyre". Q[S,A] = Q[0,1] = 1.2

e) Forklar hvorfor de to algoritmene gir forskjellig oppdatering.

Læringsalgoritmer

SARSA og Q-læring



$$Q = \begin{bmatrix} 1 & 0.5 \\ -1 & 1 \\ -2 & -1.5 \end{bmatrix}$$

Vi har en agent med følgende midlertidig Q-tabell, der hver rad tilsvarer en tilstand, henholdsvis tilstand S_0, S_1 og S_2, og hver kolonne tilsvarer et valg, henholdsvis venstre og høyre. Her er for eksempel Q-verdien til å være i tilstand S_0 (første rad) og gå til venstre (første kolonne) Q[0, 0] = 1 (med indeksering som starter på 0), mens Q-verdien for å være i tilstand S_2 (tredje rad) og gå til høyre (andre kolonne) er Q[2, 1] = −1.5.

Q-tabell oppdateringsregelen for SARSA: $Q[S,A] \leftarrow Q[S,A] + \alpha(R + \gamma Q[S',A'] - Q[S,A])$

Q-tabell oppdateringsregelen for Q-læring: $Q[S,A] \leftarrow Q[S,A] + \alpha(R + \gamma \max_a Q[S',a] - Q[S,A])$

Nytt: Anta istedenfor at agenten gikk **til høyre to ganger fra tilstanden S0**. Bruk den **originale Q-tabellen**, ikke den oppdaterte.

Beregn nå:

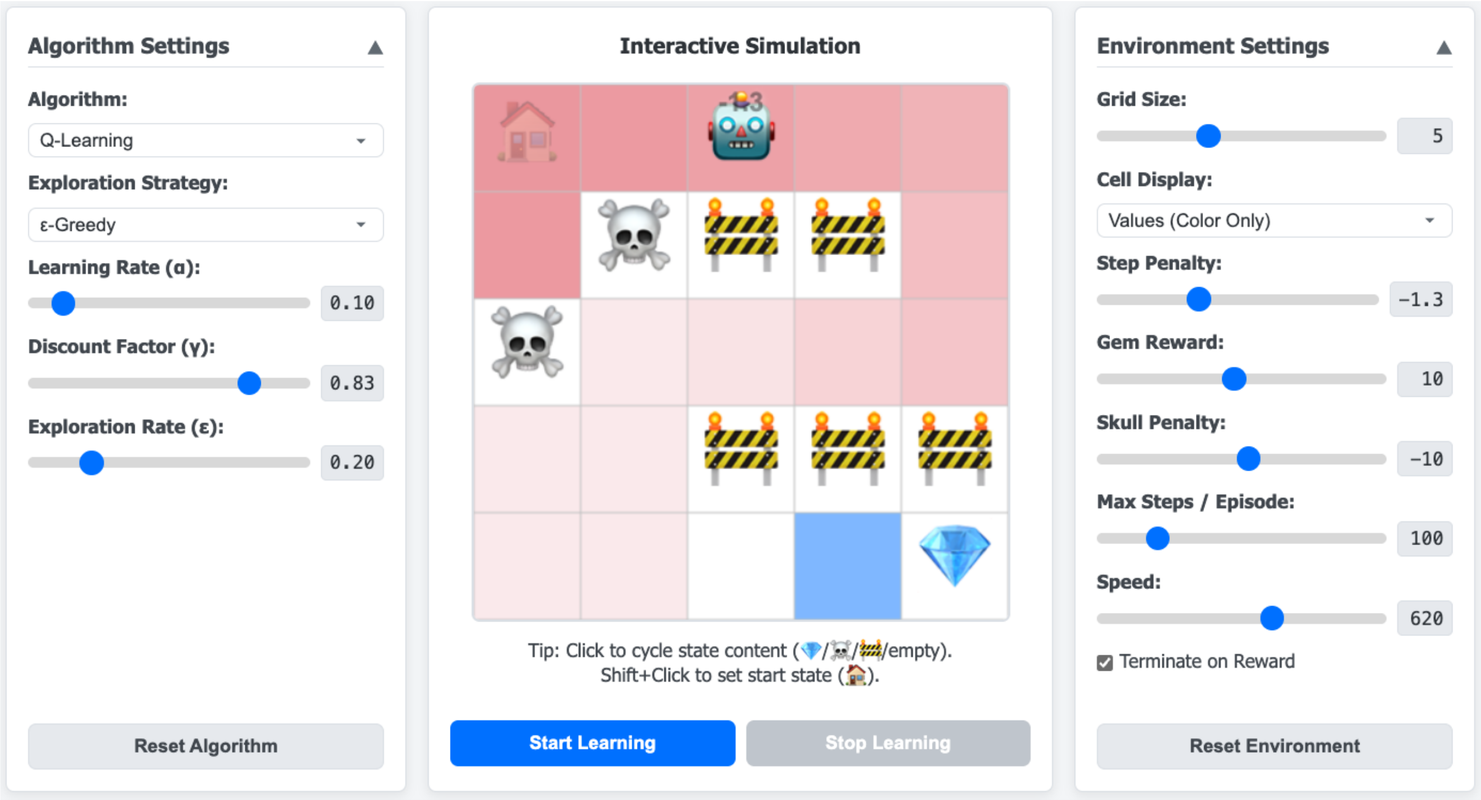
f) Bruk oppdateringsregelen til SARSA til å oppdatere Q-tabellen for tilstand S_0 og handling "høyre". Q[S,A] = 1.2

g) Bruk oppdateringsregelen til Q-læring til å oppdatere Q-tabellen for tilstand S_0 og handling "høyre". Q[S,A] = 1.2

h) Forklar hvorfor algoritmene nå gir det samme svaret.

Introduksjon til “The Beginners RL Playground”

Interaktivt verktøy for å visualisere RL i sanntid



https://awjuliani.github.io/web-rl-playground/?algo=q-learning&strategy=epsilon-greedy&lr=0.1&discount=0.83&epsilon=0.2&beta=1&actorLr=0.1&criticLr=0.1&srMLr=0.1&srWLr=0.1&grid=5&display=values-color&stepPenalty=-1.4&gemReward=10&badReward=-10&maxSteps=100&speed=910&terminate=1

Oppgave: Utforsk miljødesign

Oppgave: Lag tre forskjellige miljøer

1. Et veldig lett miljø (kun en belønning, ingen vegger eller straffer). Hvor fort lærer agenten? Hvilken rute tar den?
2. Et miljø med hindringer (vegger blokkerer direkte vei til målet). Finner agenten den beste ruten? Hvordan er læringshastigheten sammenlignet med det enkle miljøet?
3. Et farlig miljø (med både vegger og straffer). Unngår agenten straffene? Hvor stabil er oppførselen?

Diskuter i grupper:

1. Hvordan påvirker miljøet hvor raskt agenten lærer?
2. I hvilke miljøer presterer agenten godt, og hvor strever den?

Oppgave: Effekt av diskonteringsfaktor

Oppgave:

1. Bruk det vanskelige miljøet og eksperimenter med ulike verdier av diskonteringsfaktoren γ (for eksempel $\gamma = 0.1$ og $\gamma = 0.99$).

Diskuter i grupper:

1. Hva er diskonteringsfaktoren γ, og hva styrer den?
2. Hvordan endrer agentens oppførsel seg når γ er lav vs høy?
3. Velger agenten en trygg eller risikabel rute?

Oppgave: Utforsking vs. Utnyttelse

Oppgave:

1. Bruk det vanskelige miljøet og sammenlign to strategier: greedy og ε-greedy. Før dere starter, sett diskonteringsfaktoren tilbake til standardverdien $\gamma = 0.83$.

Spørsmål:

1. Hva er forskjellen mellom greedy og ε-greedy strategier?
2. Hvilken strategi konvergerer raskest når alle parameterne er like (med unntak av Exploration Rate)?
3. Når trenger agenten mye utforsking?
4. Hva skjer når ε er: 0.9, 0.1, 0?

Oppgave: Juster læringsrate α

Oppgave:

1. Bruk det vanskelige miljøet og eksperimenter med ulike verdier av læringsraten α (for eksempel α = 1 og α = 0.01). Før dere starter, sett diskonteringsfaktoren tilbake til γ = 0.83 og ε = 0.2.

Diskuter:

1. Hva er læringsraten α, og hva styrer den?
2. Hvilke verdier fører til ustabil læring?
3. Hvordan påvirker α hvor raskt agenten lærer?

Oppgave: Lur agenten

Lag et miljø der agenten gjør noe feil eller rart fordi belønningsfunksjonen eller miljødesignet er dårlig

1. Plasser ut flammer, vegger og belønninger.
2. Endre belønningene slik at agenten lærer noe *uønsket*.
3. Kjør minst to algoritmer (f.eks. Q-learning og SARSA).
4. Beskriv hva agenten "tror" om miljøet basert på oppførselen.

Eksempler:

Skap et miljø der agenten lærer:

- å gå i døden
- å unngå målet
- å gå i sirkel
- å ta en lang omvei
- å gjøre samme feil gjentatte ganger

Jobb med partner:

1. Beskriv miljøet dere lagde
2. Hvilken algoritme dere brukte
3. Hva agenten gjorde
4. Hvordan belønningsfunksjonen "lurte" agenten
5. Hva dere lærte om RL og belønning